Classic ML Interview Cheat Sheet

Oct 23, 2026 · @Mukundan

1. Foundations and Types of Learning

Machine learning builds models that learn patterns from data instead of following hand-written rules.

Types of learning

- Supervised: trained on labeled data (input plus known answer). Learns a mapping from features to a target. Covers classification and regression.
- Unsupervised: no labels. Finds structure on its own. Covers clustering and dimensionality reduction.
- Reinforcement: an agent acts in an environment and learns from rewards and penalties. Learns a policy that maximizes long-term reward.

Core terms

- Features (X): the input variables. Also called predictors or independent variables.
- Label / target (y): what you predict. The dependent variable.
- Model: the function mapping features to a prediction.
- Training vs inference: training fits the parameters to data, inference uses the fitted model to predict.

Classification vs regression

- Classification predicts a category (spam or not, fraud or legit).
- Regression predicts a continuous number (price, temperature).

Parametric vs non-parametric

- Parametric: fixed number of parameters, fixed-shape function (linear and logistic regression). Fast, needs assumptions, can underfit.
- Non-parametric: parameters grow with the data, more flexible (KNN, decision trees). Fits complex patterns, heavier, risks overfitting.

Generative vs discriminative

- Discriminative: models the class boundary directly, P(y given X) (logistic regression, SVM).

- Generative: models how the data is generated, then applies Bayes (naive Bayes).

Interview angle: they often open with "supervised vs unsupervised" or "is this classification or regression" to check you frame the problem correctly before naming any algorithm.

2. Bias-Variance Tradeoff

The single most-asked ML theory topic. Total error splits into three parts:

- **Bias:** error from wrong assumptions. The model is too simple to capture the real pattern. High bias means underfitting.
- **Variance:** error from sensitivity to the training data. The model memorizes noise and changes a lot if you swap the data. High variance means overfitting.
- **Irreducible error:** noise in the data itself. You cannot remove it.

The tradeoff: making a model more complex lowers bias but raises variance, and vice versa. The goal is the sweet spot with lowest total error, not zero of either.

Underfitting (high bias)

- Symptoms: poor performance on both training and test data.
- Fixes: more complex model, add features, reduce regularization, train longer.

Overfitting (high variance)

- Symptoms: great on training data, poor on test data (big gap between the two).
- Fixes: more training data, simpler model, regularization, dropout, early stopping, cross-validation, feature selection.

How to diagnose: compare training error and validation error.

- Both high and close: underfitting (high bias).
- Training low, validation much higher: overfitting (high variance).
- Both low and close: good fit.

Interview angle: expect "your model does 98% on train but 70% on test, what's happening and what do you do." Answer: overfitting, then list fixes. Naming the train/test gap as the signal is what they want to hear.

3. Data Splitting and Validation

The three splits

- Training set: the model learns its parameters here.
- Validation set: used to tune hyperparameters and pick models. The model never trains on it.
- Test set: touched once, at the very end, to estimate real-world performance. Looking at it during development poisons it.
- Typical split: 60/20/20 or 70/15/15. Large datasets can give more to training.

Cross-validation

- K-fold: split data into k parts, train on k-1 and validate on the held-out one, rotate, average the scores. Common k is 5 or 10.
- Why: a single split can be lucky or unlucky. CV gives a more reliable estimate and uses all data for both training and validation.
- Stratified k-fold: keeps the class ratio the same in every fold. Use it for imbalanced classification.

Data leakage (the classic trap) Leakage is when information from outside the training set sneaks into training, so the model looks great in testing but fails in production.

Common causes:

- Scaling or imputing using statistics computed on the whole dataset before splitting. Fit the scaler on train only, then apply to val and test.
- A feature that is actually a proxy for the target or only available after the outcome is known.
- Time series split at random instead of chronologically, letting the model see the future.
- The same entity (customer, patient) appearing in both train and test.

Prevention: split first, then do all preprocessing inside a pipeline fit on training data only.

Interview angle: leakage is a favorite. If you say "I standardized the features then split the data," a sharp interviewer will catch it. Always split before fitting any transform.

4. Linear and Logistic Regression

Linear regression

- What: predicts a continuous value as a weighted sum of features. y = w0 + w1x1 + w2x2 + ...
- Cost function: Mean Squared Error (MSE). Penalizes large errors heavily.
- How it fits: minimize MSE, via gradient descent or a closed-form normal equation.
- Key assumptions: linear relationship, independent errors, constant error variance (homoscedasticity), normally distributed residuals, little multicollinearity.
- Use when: target is continuous and roughly linearly related to features.

Logistic regression

- What: a classification algorithm despite the name. Predicts the probability of a class.
- How: takes the linear combination and passes it through the sigmoid function, which squashes any number to a 0 to 1 probability.
- Decision boundary: predict class 1 if probability > threshold (default 0.5). The boundary itself is linear.
- Cost function: log loss (binary cross-entropy), not MSE. MSE here is non-convex and would get stuck.
- Odds and log-odds: logistic regression is linear in the log-odds (logit). A coefficient tells you how the log-odds change per unit of that feature.
- Multiclass: softmax (multinomial) or one-vs-rest.

```
\sigma(z) = \frac{1}{1 + e^{-z}}
```

Interview angle: two classics. "Why not use MSE for logistic regression" (non-convex, log loss is convex and matches the probabilistic setup). "Why is it called regression if it classifies" (it regresses the log-odds, then thresholds). You have built both in your bootcamp, so lean on that.

5. Gradient Descent and Cost Functions

Cost function: a single number measuring how wrong the model is across the data. Training means finding the parameters that minimize it.

Gradient descent: the algorithm that does the minimizing. It computes the gradient (slope) of the cost with respect to each parameter and steps in the opposite direction, repeatedly, until it reaches a minimum. Think of walking downhill.

Learning rate: the step size.

- Too high: overshoots, bounces around, may diverge.
- Too low: converges very slowly.
- A core hyperparameter to tune.

Variants

- Batch GD: uses the whole dataset per step. Stable but slow and memory-heavy.
- Stochastic GD (SGD): one random sample per step. Fast and noisy, the noise can escape shallow local minima.
- Mini-batch GD: a small batch per step. The practical default, balancing speed and stability.

Convex vs non-convex

- Convex (one bowl, one minimum): linear and logistic regression. Gradient descent reliably finds the global minimum.
- Non-convex (many hills and valleys): neural networks. Can land in local minima or saddle points, which is why initialization and optimizers matter.

Convergence issues: diverging (learning rate too high), getting stuck (saddle points, flat regions), slow progress (poor scaling, so feature scaling helps a lot here).

Interview angle: "what's the role of the learning rate" and "batch vs stochastic vs mini-batch" are common. Mentioning that feature scaling speeds up convergence shows practical depth.

6. Regularization

Why: to fight overfitting. It adds a penalty on large coefficients to the cost function, pushing the model toward simpler weights that generalize better.

L2 (Ridge)

- Penalty: sum of squared weights.
- Effect: shrinks all weights toward zero but rarely exactly to zero. Keeps all features.
- Good default, handles correlated features gracefully.

L1 (Lasso)

- Penalty: sum of absolute weights.
- Effect: drives some weights exactly to zero, so it performs automatic feature selection.
- Good when you suspect many features are useless and want a sparse model.

Elastic Net: a mix of L1 and L2. Gets sparsity plus stability with correlated features.

Lambda (regularization strength)

- Higher lambda: stronger penalty, simpler model, more bias, less variance.
- Lower lambda: weaker penalty, closer to plain regression.
- Set it with cross-validation.

The key distinction to memorize: L1 produces sparsity (feature selection) because its penalty has sharp corners at zero, so the optimum often lands exactly on zero. L2 only shrinks, because its penalty is smooth.

Beyond linear models, the same idea appears as dropout and early stopping in neural networks, and pruning and max-depth limits in trees.

Interview angle: the near-certain question is "L1 vs L2, and which one does feature selection." Answer: L1 (Lasso), and explain the corners-at-zero intuition. That one sentence separates memorizers from people who understand it.

7. Decision Trees and Ensembles

Decision tree

- What: splits data by asking yes/no questions on features, forming a tree. Each leaf gives a prediction.
- How it splits: picks the split that most reduces impurity. Gini impurity or entropy (information gain) for classification, variance reduction for regression.
- Pros: interpretable, needs no feature scaling, handles non-linear patterns.
- Cons: overfits easily (a deep tree memorizes). Controlled by max depth, min samples per leaf, and pruning.

Ensembles: combine many weak models into one strong one. Two families.

Bagging (bootstrap aggregating)

- Train many models in parallel on random samples of the data, then average or vote.
- Reduces variance. Random Forest is bagging on decision trees, plus random feature subsets at each split to decorrelate the trees.

Boosting

- Train models sequentially, each one correcting the errors of the previous.
- Reduces bias. Examples: AdaBoost, Gradient Boosting, XGBoost, LightGBM.

- XGBoost is the competition and interview favorite: regularized gradient boosting, fast, handles missing values.

Random Forest vs Gradient Boosting (know this cold)

Aspect	Random Forest	Gradient Boosting
Trees built	In parallel, independent	Sequentially, each fixes the last
Mainly reduces	Variance	Bias
Overfitting risk	Lower, robust	Higher, needs careful tuning
Speed to train	Faster, parallelizable	Slower
Tuning effort	Forgiving	Sensitive (learning rate, depth, estimators)
Typical accuracy	Strong	Often higher if tuned well

Interview angle: "bagging vs boosting" and "random forest vs XGBoost" come up constantly. The one-liner: bagging reduces variance by averaging independent trees, boosting reduces bias by correcting errors in sequence.

8. Distance and Probability Models

K-Nearest Neighbors (KNN)

- What: to classify a point, look at its k closest neighbors and take the majority vote (or average for regression).
- Lazy learner: no training phase, it just stores the data and does all the work at prediction time.
- Choosing k: small k is noisy and overfits, large k is smooth and can underfit. Odd k avoids ties.
- Needs feature scaling, since it is distance-based.
- Weakness: the curse of dimensionality. In high dimensions all points look equally far apart, so distances stop being meaningful. Also slow at prediction on big data.

Support Vector Machine (SVM)

- What: finds the hyperplane that separates classes with the largest margin (widest gap to the nearest points, the support vectors).

- Kernel trick: maps data into a higher dimension so a linear boundary can separate data that was not linearly separable, without computing that mapping explicitly. Common kernels: linear, polynomial, RBF.
- Strong in high dimensions and with clear margins. Slower on very large datasets, sensitive to scaling.

Naive Bayes

- What: a probabilistic classifier using Bayes theorem.
- The naive assumption: all features are conditionally independent given the class. Rarely true, but works surprisingly well.
- Fast, needs little data, strong baseline for text classification and spam filtering.

```
P(y \mid X) = \frac{P(X \mid y)\, P(y)}{P(X)}
```

Interview angle: "why does KNN need scaling" (distance-based), "what is the kernel trick" (implicit higher-dimensional separation), and "why is naive Bayes called naive" (the independence assumption) are the three near-guaranteed questions here.

9. Unsupervised Learning

K-Means clustering

- What: groups data into k clusters. Each point joins the nearest cluster center, then centers move to the mean of their points, repeat until stable.
- Choosing k: the elbow method (plot within-cluster error vs k, pick the bend) or the silhouette score.
- Limitations: you must pick k upfront, it assumes round equal-sized clusters, it is sensitive to initialization (k-means++ helps) and to outliers. Needs scaling.

Hierarchical clustering

- Builds a tree of clusters (dendrogram) by repeatedly merging the closest pair. No need to pick k in advance, you cut the tree where you like. Slower on large data.

DBSCAN

- Density-based. Groups points packed closely together and labels sparse points as noise. Finds arbitrary shapes and does not need k, but struggles when clusters have very different densities.

PCA (Principal Component Analysis)

- What: dimensionality reduction. Finds new axes (principal components) along the directions of greatest variance, then keeps the top few, dropping the rest.
- Why: fewer features, less noise, faster training, and it lets you visualize high-dimensional data in 2D or 3D.
- Variance explained: each component captures some percentage of the total variance. You keep enough components to retain, say, 95%.
- Notes: components are linear combinations of original features, so you lose interpretability. Always scale first. This was in your bootcamp alongside SMOTE and pipelines, so you can speak to it from hands-on use.

Interview angle: "how do you choose k in k-means" (elbow or silhouette), "what does PCA actually do" (projects onto max-variance directions), and "supervised vs unsupervised, give an example of each" are the common ones.

10. Feature Engineering and Preprocessing

Often the biggest lever on model performance. Good features beat a fancy algorithm.

Scaling (making features comparable in range)

- Standardization (Z-score): subtract mean, divide by standard deviation. Centers at 0. The common default.
- Normalization (min-max): rescales to a 0 to 1 range.
- Who needs it: distance and gradient-based models (KNN, SVM, k-means, logistic/linear regression, neural nets, PCA).
- Who does not: tree-based models (decision trees, random forest, XGBoost) are scale-invariant.
- Always fit the scaler on training data only, then apply to validation and test (see leakage, section 3).

Encoding categorical variables

- One-hot encoding: one binary column per category. Use for nominal (unordered) categories. Watch out for high-cardinality blowup.
- Label / ordinal encoding: integers per category. Only for ordered categories (low/medium/high), since numbers imply order.

Missing values

- Drop rows or columns if few and non-informative.

- Impute: mean/median for numeric, mode or a "Missing" category for categorical.
- Fit imputation on training data only.

Imbalanced data (your bootcamp covered this)

- Resampling: SMOTE (synthesizes new minority examples), ROSE, random over/undersampling.
- Class weights: tell the model to penalize minority errors more. Often the cleanest option.
- Pick the right metric: accuracy is useless here, use precision, recall, F1, or PR-AUC.

Interview angle: "which algorithms need feature scaling" (trees do not, distance and gradient models do) and "your dataset is 95% one class, what do you do" (resample or class-weight, and change the metric) are standard. Naming SMOTE plus the metric change signals real experience.

11. Model Evaluation Metrics

Confusion matrix (the foundation for classification)

- True Positive (TP): predicted yes, was yes. True Negative (TN): predicted no, was no.
- False Positive (FP): predicted yes, was no (false alarm, Type I error).
- False Negative (FN): predicted no, was yes (a miss, Type II error).

Classification metrics

- Accuracy = (TP + TN) / all. Misleading on imbalanced data: predict all-majority and still score 95%.
- Precision = TP / (TP + FP). Of the ones you flagged, how many were right. Care about this when false positives are costly (spam flags, fraud alerts).
- Recall (sensitivity) = TP / (TP + FN). Of the real positives, how many you caught. Care about this when false negatives are costly (disease, fraud you must not miss).
- F1 = harmonic mean of precision and recall. One number when you need balance, especially on imbalanced data.
- Precision-recall tradeoff: lowering the decision threshold raises recall and lowers precision, and vice versa. You tune the threshold to the business cost, you do not have to use 0.5.

ROC-AUC vs PR-AUC

- ROC curve plots true positive rate vs false positive rate across thresholds. AUC is the area under it: 1.0 perfect, 0.5 random.
- On heavily imbalanced data ROC-AUC looks too optimistic. PR-AUC (precision vs recall) is the better read there.

Regression metrics

- MAE: mean absolute error. Same units as target, robust to outliers.
- MSE: mean squared error. Punishes big errors harder, in squared units.
- RMSE: square root of MSE. Back in target units, the common default.
- R-squared: fraction of variance explained, 0 to 1. 0.8 means the model explains 80% of the variation.

Interview angle: the staple is "precision vs recall, and when do you favor each." Tie it to cost: cancer screening favors recall (do not miss a case), spam filtering favors precision (do not trash real mail). Add "why accuracy fails on imbalanced data" and you have covered what they probe most. This is where your QE and release-governance background genuinely stands out.

12. Hyperparameter Tuning and the ML Workflow

Parameters vs hyperparameters

- Parameters: learned from data during training (the weights in regression, the splits in a tree).
- Hyperparameters: set by you before training (learning rate, k in KNN, tree depth, lambda, number of estimators).

Tuning methods

- Grid search: try every combination in a defined grid. Thorough but expensive, blows up with more parameters.
- Random search: sample random combinations. Often finds a good setting faster than grid search, especially when only a few parameters matter.
- Bayesian optimization: uses past results to choose the next combination intelligently. More efficient for expensive models (Optuna, Hyperopt).
- Always tune using cross-validation on the training/validation data, never the test set.

End-to-end ML workflow (a clean answer for "walk me through an ML project")

1. Frame the problem: classification or regression, and the metric that matches the business goal.
2. Get and split the data: train/val/test, before any preprocessing.
3. Explore (EDA): distributions, missing values, correlations, class balance.
4. Preprocess and engineer features: scale, encode, impute, all inside a pipeline fit on train only.
5. Baseline model: start simple (logistic regression) to set a reference.
6. Train and compare models: cross-validate, tune hyperparameters.
7. Evaluate on the untouched test set with the chosen metric.
8. Deploy and monitor: watch for drift and retrain when performance drops.

Interview angle: "grid vs random search" and "walk me through how you would approach an ML problem" are common closers. Hitting the workflow steps in order, and mentioning that monitoring and retraining come after deployment, shows you think about production, which is exactly where your QE and MLOps background pays off.

This covers Classic ML. Deep Learning, Transformers/LLMs, and MLOps can each get their own sheet when you are ready.